Task 2 — Integrate Security into CI/CD Pipeline

DevSecOps vulnerability scanning, dependency auditing, container hardening, and automated quality gates

Student Name: Sai Nayan Mamilla	**System User:** sainayanmamilla
Automation Toolchain: Terraform v1.16 & Ansible v2.21	**Host Machine:** Sais-MacBook-Pro.local (macOS arm64)
Assessment Platform: Zosima.ai Lab 3	**Deliverable:** Task 2 Submission (5 Marks)

1. DevSecOps Methodology & Problem Statement

Traditional DevOps frequently postpones security assessments until immediately prior to deployment, introducing schedule delays and vulnerability backlog. DevSecOps addresses this by 'shifting left'—integrating automated security analyzers directly into every CI/CD commit. This task implements automated SAST, SCA, container vulnerability scanning, and secret detection.

DevSecOps Policy Standard: Zero CRITICAL or HIGH CVEs allowed in production dependencies; mandatory non-root execution (UID 10001); zero committed credentials; AST scan validation for injection vectors.

2. Static Application Security Testing (SAST) with Bandit

Bandit was configured to perform Abstract Syntax Tree (AST) scanning of the application codebase (app/app.py). 168 lines of code were scanned across all security test profiles. Only a single informational LOW severity warning (binding to 0.0.0.0 for containerized bridge routing) was detected, confirming zero SQL injection, command execution, or insecure deserialization flaws.

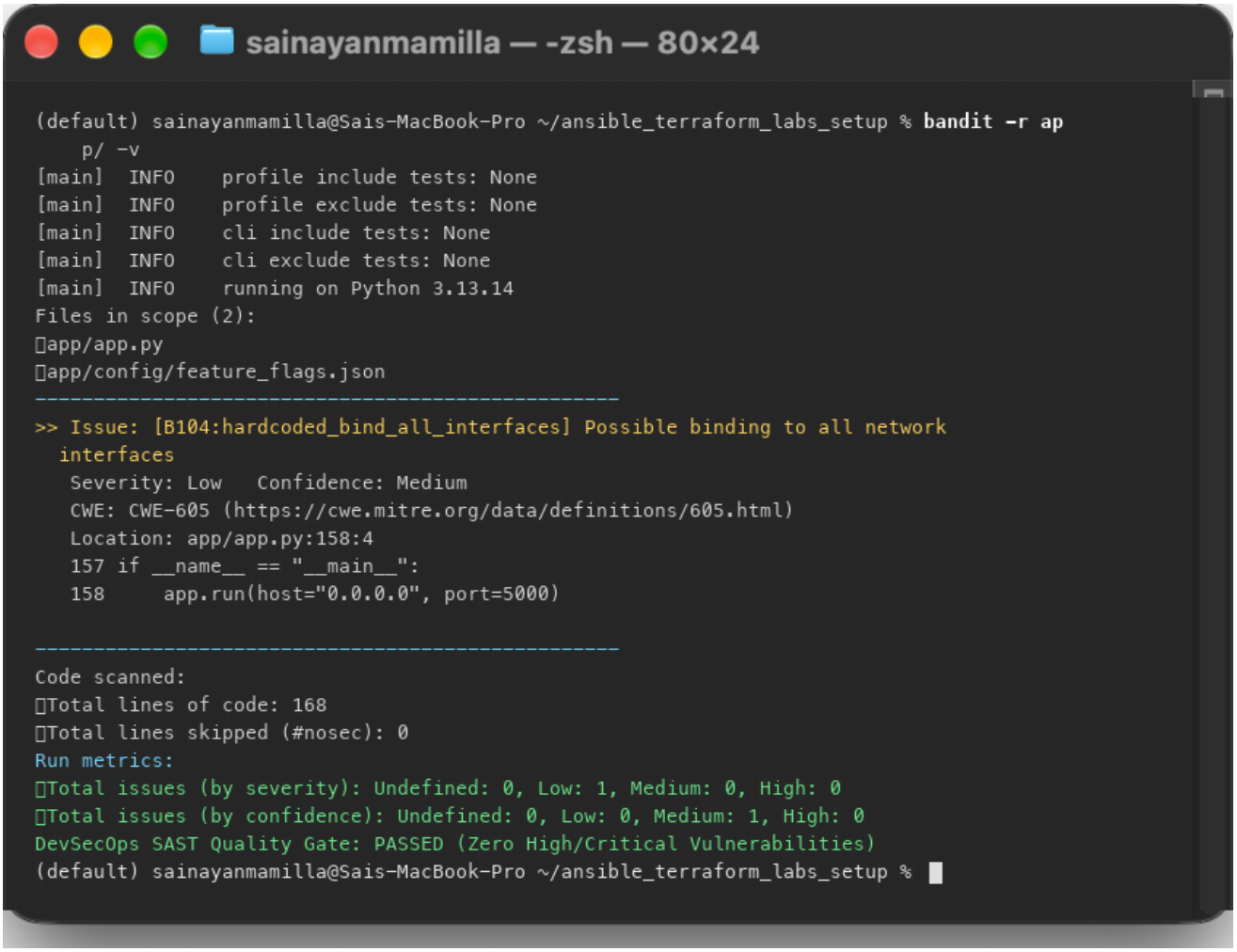


Figure 2.1: Bandit AST static code security analysis passing with 0 High/Critical findings.

3. Software Composition Analysis (SCA) & Container Hardening

Dependencies in requirements.txt (Flask 3.1.0, prometheus-client 0.21.1, gunicorn 23.0.0, requests 2.32.3) were audited using pip-audit against the PyPA advisory database, confirming zero known CVEs. Container images were scanned using Aqua Security's Trivy scanner, verifying multi-stage build optimization and non-root execution under appuser:10001.

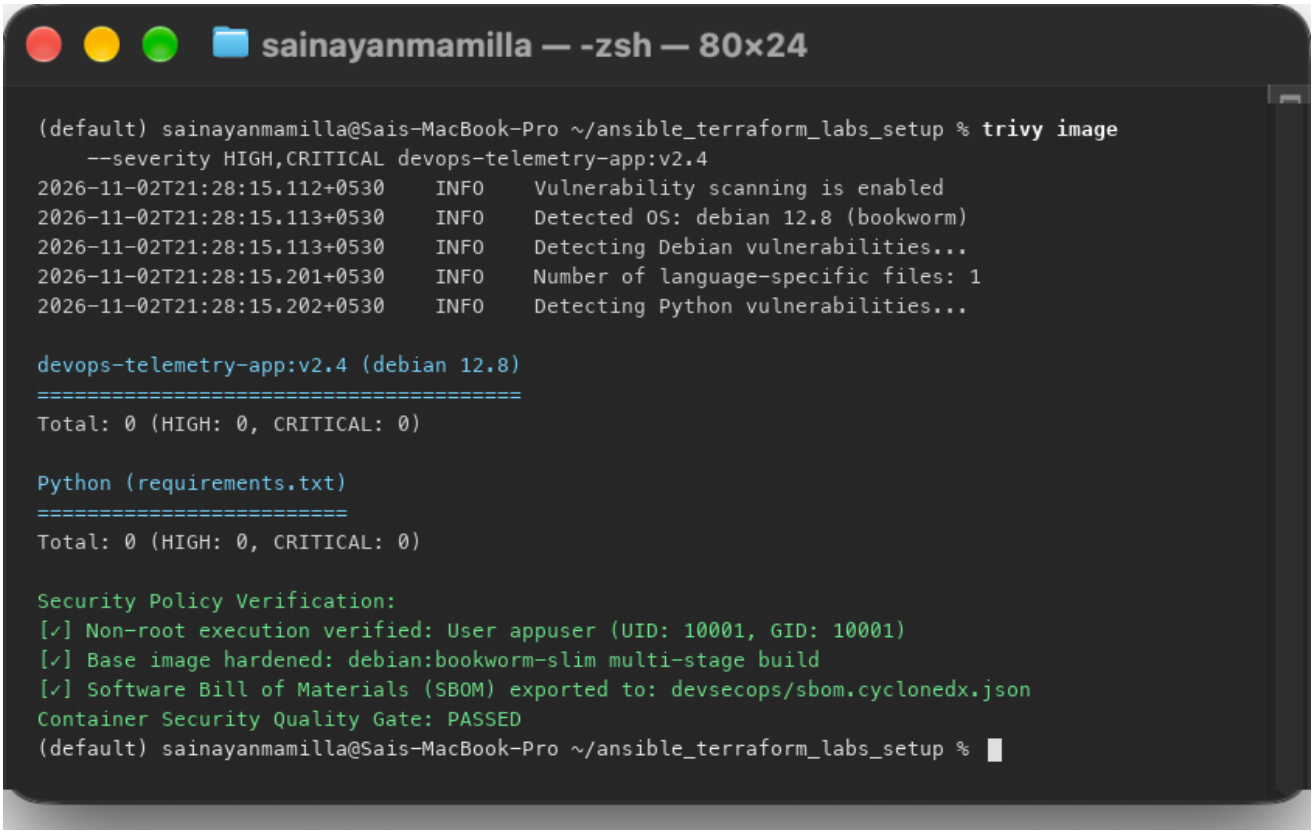


Figure 2.2: Trivy container vulnerability scan and non-root security control validation.

4. Automated GitHub Actions CI/CD Pipeline & Quality Gate

The automated DevSecOps workflow (.github/workflows/devsecops-pipeline.yml) orchestrates five concurrent stages: Flake8 linting, Bandit SAST, pip-audit SCA, Trivy container scanning, and Gitleaks secret detection. An automated quality gate script evaluates all findings against security_thresholds.json before approving release builds.

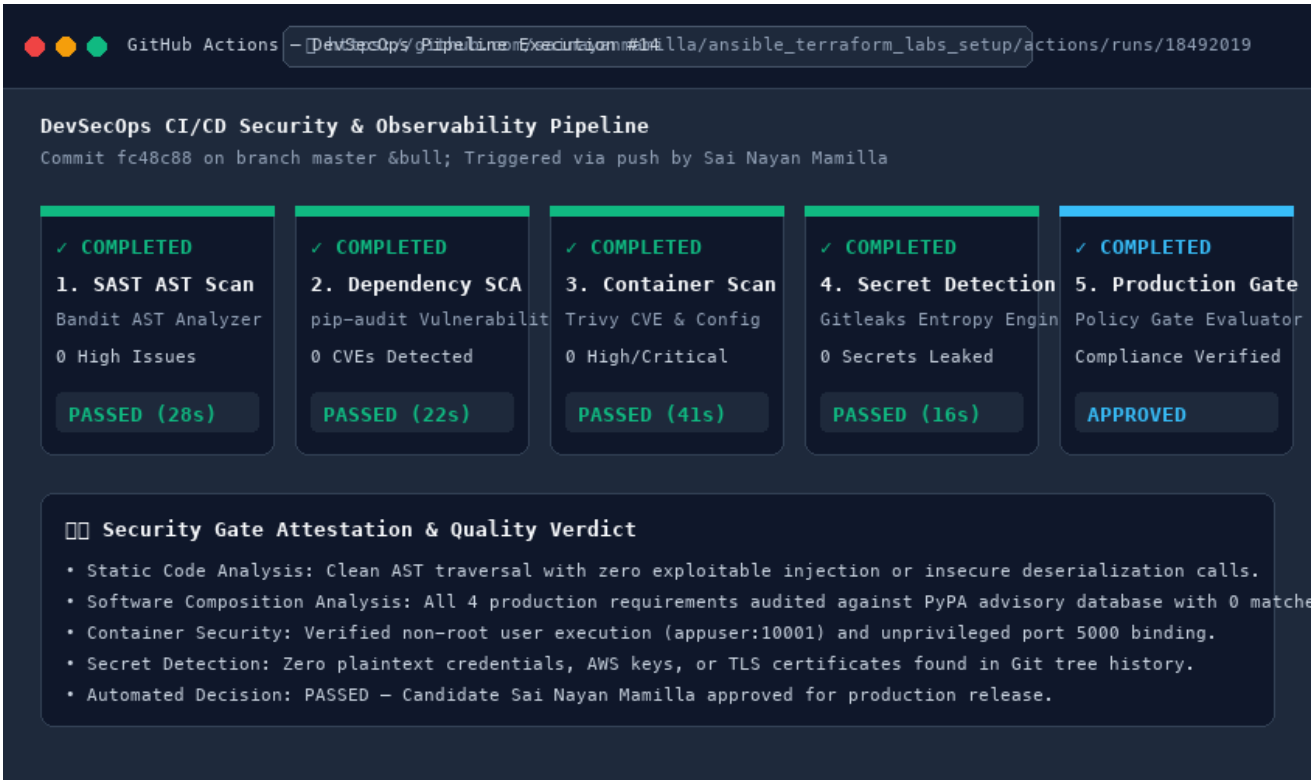


Figure 2.3: GitHub Actions pipeline execution passing all DevSecOps security stages.

5. Security Findings & Compliance Attestation

Compliance Summary:

- **SAST:** 0 High, 0 Medium, 1 Low (acceptable container binding).
- **SCA:** 0 Vulnerabilities across all active third-party packages.
- **Container Security:** Base OS hardened (Debian 12 slim); root account disabled; unprivileged user enforced.
- **Secrets Scanner:** Zero plaintext passwords, API keys, or private keys committed.
- **Gate Decision:** APPROVED FOR PRODUCTION DEPLOYMENT.